ACCESCO LIVING

Dual Replenishment Engine

Model Selection, System Design & Detailed Data Requirements

Warehouse replenishment: TimesFM-3 forecasting + deterministic inventory planning
Customer-facing replenishment: XGBoost Survival AFT time-to-depletion

Document purpose. Specify the selected model for each engine, how each model will be used, the exact data required to build and validate it, the operational decision logic around it, and a phased implementation plan that respects Accesco's current data constraints.

Decision status: Model architecture selected; production training and final deployment are conditional on data availability, validation, licensing, and operational sign-off.

Prepared 10 October 2026 | Version 2.0 — customer model updated to XGBoost Survival AFT

At a glance

Engine	Selected model / method	Primary prediction	What turns prediction into action
Warehouse	**TimesFM-3** as the demand-forecasting model, with an explicit inventory/replenishment planning layer.	Probabilistic demand forecast for each SKU and warehouse/dark-store time series over a planning horizon.	Inventory position + lead time + safety stock + shelf life + pack/MOQ + transfer rules → recommended order/transfer quantity and timing; staff approval required.
Customer-facing	**XGBoost Survival AFT** for time-to-event prediction, with global product patterns and onboarding signals during cold start, then increasing personalisation as evidence accumulates.	Distribution of time until a product is likely to run out, not merely time until the next purchase.	Prediction uncertainty + feedback + notification policy → reminder window and channel, with push as default and WhatsApp only by explicit opt-in.

Contents

• 1. Executive decision and operating constraints
• 2. Shared architecture and data foundation
• 3. Warehouse engine: TimesFM-3 and replenishment planner
• 4. Customer engine: XGBoost Survival AFT
• 5. Detailed data dictionary: warehouse, customer and shared data
• 6. Event definitions, labels and feature engineering
• 7. Cold-start, feedback and household handling
• 8. Outputs, APIs, workflow and human controls
• 9. Data quality, validation, monitoring and retraining
• 10. Implementation plan, risks and acceptance criteria
• 11. References and implementation notes

Important scope note: This document does not claim that the models can be trained immediately. Accesco currently has SKU identifiers/catalogue and finalized locations, but lacks the historical transaction, inventory, supplier lead-time and customer purchase/consumption history required for reliable production training. The first deliverable is therefore the data/event pipeline and a transparent rule-based bootstrap, followed by model training when evidence is sufficient.

1. Executive decision and operating constraints

Accesco needs two connected but distinct engines. The warehouse engine protects service levels and inventory economics across the central warehouse and dark stores. The customer-facing engine helps a person or household anticipate when a tracked product is likely to be depleted and reminds them in time. They share product identity and some catalogue data, but they predict different outcomes and must not be collapsed into one model.

1.1 Decisions locked for this design

- **Warehouse:** use TimesFM-3 as the selected demand-forecasting candidate. Do not ask the forecast model to decide final order quantities by itself. A separate inventory planner converts forecasts into replenishment recommendations.
- **Customer:** use XGBoost Survival AFT to estimate time to likely depletion. The target is not simply the next purchase date.
- **Cold start:** use global product/category consumption evidence and the onboarding survey. As the customer accumulates evidence, personalize increasingly. The working policy is to begin shifting toward personal data after 3–5 purchases of that product, provided those observations are usable and reasonably consistent.
- **Households:** support shared household products and separately tracked personal products.
- **Feedback:** capture optional states such as “Still plenty,” “Running low,” “Finished,” and remaining quantity where practical.
- **Delivery:** in-app/push is the default reminder channel. WhatsApp is optional and must require explicit opt-in.
- **Approval:** all warehouse purchase and transfer recommendations require staff review/approval; no automatic supplier order is implied by this document.
- **Operating cadence:** warehouse monitoring is daily; customer predictions update when new purchases, feedback, or other meaningful events arrive and can be refreshed on a scheduled cadence.

1.2 Current data position

Data area	Current known position	Implication
Product catalogue	SKU/product identifiers exist; catalogue attributes may be incomplete.	Can define keys and start validating catalogue completeness. Confirm units, pack sizes, shelf life and category fields.
Locations	Warehouse/dark-store locations are finalized.	Can define location IDs and inventory scope; still need location-SKU stock and movement events.
Warehouse transactions	Historical order/demand history is not currently available in the project context.	Cannot establish a trustworthy SKU × location demand history until order-line events are collected or imported.
Inventory & supply	Inventory records, supplier lead-time history and purchase-order history are not currently available in the project context.	Replenishment arithmetic can be designed now, but operational outputs remain provisional until stock, on-order, reservation and lead-time fields exist.
Customer history	No established customer accounts/order history is available for model training.	Start with onboarding and product/category priors; collect purchases and depletion feedback prospectively.
Platform choices	Final web/app, backend, database and ML deployment decisions are not confirmed.	Keep event contracts and model interfaces technology-neutral until architecture decisions are made.

1.3 What “best model” means here

The selection is for the intended job and known constraints, not a promise of accuracy before evaluation. TimesFM-3 is assigned to the warehouse demand-forecasting task; XGBoost Survival AFT is assigned to the customer time-to-depletion task. Business rules, stock constraints, event semantics, confidence calibration, validation and human approval remain necessary around both models.

2. Shared architecture and data foundation

2.1 End-to-end architecture

Layer	Responsibilities	Core records/events
Operational source systems	Capture orders, order lines, inventory movements, purchase orders, receipts, transfers, product catalogue and customer feedback.	order_created / order_line; stock_adjusted; PO_created / PO_received; transfer_dispatched / transfer_received; customer_purchase; depletion_feedback.
Data ingestion and event log	Store immutable timestamped events with source, event time, ingestion time, stable IDs and correction/reversal links.	Raw events + change history; idempotency key; event schema version.
Canonical data model	Resolve product/SKU, location, household, customer and unit-of-measure identity. Enforce consistent timestamps and quantity units.	Product dimension; location dimension; customer/household; transaction facts; inventory snapshots and movements.
Feature and label pipelines	Build daily SKU-location demand series for warehouse forecasting and customer-product observation windows for survival modelling.	Warehouse daily panel; customer-product landmark rows; censored/event-time labels.
Model services	Generate warehouse forecast distributions and customer depletion-time distributions.	Forecast by SKU/location/horizon; AFT prediction parameters/time quantiles.
Decision engines	Convert predictions into stock recommendations or reminder windows with explicit constraints.	Replenishment recommendation; reminder eligibility/schedule.
User interfaces and audit	Show rationale, uncertainty, approval state, feedback and overrides.	Dashboard/app notifications; approver ID; action outcome; model version; override reason.

2.2 Shared design principles

- **One canonical product ID:** map SKU, barcode, pack variant and customer-visible product ID. Avoid training the same product as separate identities due to inconsistent codes.
- **Event time and ingestion time:** preserve both. Late-arriving orders or corrections must not rewrite what the model would have known at an earlier prediction time.
- **Unit-of-measure discipline:** store the purchased unit and normalized base unit (e.g., pack, ml, g, piece). Record pack conversion factors and version them.
- **No future leakage:** features at prediction time must not contain future purchases, future stock corrections or feedback recorded after the prediction timestamp.
- **Traceability:** each forecast/recommendation/reminder should carry a model version, data-as-of timestamp, feature snapshot reference and reason codes.
- **Separate prediction from policy:** models estimate demand/depletion; deterministic policy layers decide stock quantities, safety buffers, reminder timing, eligibility and approvals.

2.3 Recommended data storage zones

Zone	Purpose	Minimum retention/detail
Raw event ledger	Auditable original facts from source systems.	Append-only where possible; preserve corrections and source IDs.
Canonical operational tables	Clean order lines, stock ledger, purchase orders, transfers, product/location/customer dimensions.	Current state plus historical effective dates where relevant.
Analytics/model tables	Feature-ready warehouse daily panel and customer-product survival rows.	Rebuildable from canonical events; version transformations.
Prediction/action log	What the model predicted and what the system did.	Store prediction timestamp, model version, intervals, action and subsequent outcome.

3. Warehouse engine: TimesFM-3 forecasting + replenishment planner

3.1 Job to be solved

The warehouse engine supports a central warehouse and multiple dark stores. It should forecast demand at the SKU × location level, estimate stockout exposure and recommend when and how much to replenish. Replenishment can mean supplier purchase into the central warehouse or a transfer from the central warehouse to a dark store. The model should be run on a daily monitoring cycle, with urgent re-evaluation when meaningful stock or order events occur.

3.2 Selected model: TimesFM-3

TimesFM-3 is the selected forecasting candidate because it is a pretrained time-series foundation model designed to forecast from historical sequences and supports multivariate forecasting and covariates in its current release. This is useful for a product catalogue with many SKU-location series and limited Accesco-specific history at launch. Its output must still be validated on Accesco demand, including intermittent/low-volume products, stockout periods, promotions and perishable goods. See the official model repository and deployment/licensing notes in References.

Production/licensing gate: the open-source TimesFM 3.0 checkpoint has had a non-commercial/non-production restriction for self-hosted weights, while Google Cloud's BigQuery ML implementation is documented for commercial use. Before production deployment, verify the current license for the exact checkpoint/serving route. If Accesco intends to self-host, do not assume that downloading the public 3.0 weights grants production rights; choose a commercially permitted route or a checkpoint/license that explicitly allows the intended use.

3.3 What TimesFM-3 does—and does not do

Component	Responsibility
TimesFM-3 forecast model	Predicts future demand from historical demand series and supported covariates. Where configured, use quantiles/forecast intervals to represent uncertainty.
Inventory position calculator	Computes available stock, on-order supply, reserved/committed demand, and transfer pipeline without double counting.
Replenishment planner	Uses forecast demand over lead time and review period, safety stock/service target, stock constraints, MOQ/pack sizes, shelf life and capacity to propose quantities.
Allocation/transfer planner	Determines whether a dark store should receive stock from the central warehouse or whether the central warehouse must place a supplier order. Respects stock availability, transfer lead time and expiry.
Approval workflow	Shows recommendation and rationale to an authorized staff member; captures approve/edit/reject, reason and eventual outcome.

3.4 Forecast grain and horizon

- **Primary grain:** one row per business date × SKU × location, with demand and availability fields.
- **Forecast horizon:** at least the relevant replenishment lead time plus the next review interval. Use a longer horizon where supplier minimum order quantities, seasonal planning or shelf life require it.
- **Target series:** fulfilled demand/sales quantity, not blindly the order quantity if orders were cancelled, substituted or not fulfilled.

- **Zero sales are ambiguous:** zero can mean zero demand or no availability. Include stockout/availability signals; do not interpret out-of-stock days as ordinary zero demand.
- **Aggregation:** preserve SKU × location data for local stock decisions. Category and network-level aggregates may support diagnostics and cold-start planning but must not erase local differences.
- **Forecast uncertainty:** retain forecast quantiles or calibrated prediction intervals. Replenishment should use a service-level policy and cost/expiry constraints rather than a single point forecast alone.

3.5 Replenishment planning logic

The planner is a deterministic decision layer. A forecast does not itself decide an order. At each SKU-location review, compute a consistent inventory position and compare it with demand expected during replenishment exposure plus a safety buffer.

Inventory position (conceptual): usable on-hand + confirmed on-order/inbound supply – reserved/committed units – known unavailable/damaged stock, adjusted for the precise operational definitions used in Accesco's ledger.

Reorder point (conceptual): forecast demand over the effective replenishment lead-time window + safety stock. If lead time varies, use its distribution or a conservative service-level estimate rather than a single unvalidated average.

Suggested order quantity (conceptual): target stock position – current inventory position, rounded to supplier pack/MOQ and then checked against storage capacity, shelf life, expiry, budget and demand. Negative quantities become no order; excess due to rounding must be visible to the approver.

These are planning principles, not final production formulas. Exact policy must be approved by Operations and Finance, including service-level targets, holding cost, stockout cost, spoilage limits, supplier MOQs and whether reservations are included in the inventory position.

3.6 Central warehouse and dark-store decisions

Decision	Required inputs	Expected output
Dark-store replenishment	Dark-store demand forecast; usable on-hand; reserved demand; inbound transfer; transfer lead time; store capacity; SKU shelf life.	Recommended transfer quantity, source location, departure/arrival need-by date, urgency and rationale.
Central warehouse supplier order	Network demand/transfer requirements; central stock; supplier lead time and variability; supplier MOQ/pack size; open POs; shelf life and receiving capacity.	Recommended supplier order timing/quantity, expected coverage, risk and approval status.
Allocation under shortage	Forecast demand by store; current stock; lead times; priority/SLA; expiry; transfer capacity.	Ranked allocation suggestion with fairness/service policy; requires explicit operations policy.
Perishable inventory	Expiry/lot/batch, remaining shelf life, demand during remaining life, FEFO policy and temperature/quality status where applicable.	Expiry-aware replenishment and allocation; do not recommend stock likely to expire before sale.
Exception alerts	Stockout risk, stale stock, forecast uncertainty, no supply/lead-time data, anomalous demand.	Human-review alert rather than a falsely precise automatic recommendation.

3.7 Perishables and non-perishables

For perishables, the planner must operate at lot/batch level when expiry dates exist, use FEFO (first-expiry-first-out) where applicable, and check that expected demand before expiry can absorb the proposed quantity. Shelf life should constrain order size and allocation. For non-perishables, expiry may be absent or long, but capacity, holding cost, minimum order quantity, supplier reliability and obsolescence can still constrain replenishment. Do not apply one universal safety-stock or reorder rule to every product category.

3.8 Warehouse model inputs and features

Feature family	Required attributes/examples	Why it matters
Time index	business_date; timezone; day-of-week; week; month; holidays/festivals; working day; pay-cycle or local event flags if available.	Captures calendar-driven demand patterns.
SKU identity	sku_id; category/subcategory; brand; pack size; base unit; product lifecycle/status.	Lets the series be joined to product attributes and supports new SKU handling.
Location identity	location_id; location_type (central/dark store); region/zone; store opening date; operating hours.	Separates local demand and operational context.
Demand target	fulfilled_units; ordered_units; cancelled_units; substituted_units; returns; net_units; gross_sales; data-quality flags.	Defines the actual forecasting target and exposes distortions.
Availability	opening/closing on-hand; in-stock minutes/hours; stockout flag/duration; assortment-live flag; unavailable reason.	Prevents stockouts from being mistaken for lack of demand.
Inventory/supply	receipts; transfers in/out; adjustments; reserved units; open PO quantity; expected arrival date; supplier lead time.	Supports planning and explains sales constraints; some fields are planner inputs rather than TimesFM inputs.
Commercial covariates	price; discount; promotion type/start/end; placement/visibility if tracked.	Explains demand shifts; only use covariates known at forecast time.
External covariates	weather, local events, holiday/festival, local demand drivers where available and lawful.	May explain local variability; optional and should be validated for value.
Perishability	shelf_life_days; expiry date by lot; temperature/quality hold; FEFO status.	Essential for action constraints; not necessarily a core demand input.
Data lineage	source_system; source_event_id; created_at; updated_at; ingested_at; correction/reversal; schema_version.	Supports reproducibility, audit and leakage prevention.

3.9 Warehouse output contract

Field	Meaning
sku_id, location_id, forecast_created_at	Stable identity and timestamp for the forecast.
horizon_date / horizon_step	Date or step for each forecast point.
forecast_units_p50	Central/median forecast if supported by the chosen output configuration.
forecast_units_p10 / p90 (or equivalent quantiles)	Lower/upper forecast estimates; calibration must be checked.
demand_series_quality / stockout_contamination_flag	Warnings about insufficient history, intermittent demand or periods with constrained availability.
inventory_position, lead_time_days, safety_stock, reorder_point	Planner calculations with input timestamps and policy version.
recommended_action, recommended_qty, source_location / supplier	Proposed transfer/order action and quantity; not a committed transaction.
need_by_date, expected_coverage_days, expiry_risk, stockout_risk	Operational explanation of urgency and downside risk.
confidence/status, reason_codes, model_version, planner_version	Human-readable caveats and full audit trace.
approval_status, approver_id, decision_timestamp, override_reason	Approval and override trail.

4. Customer engine: XGBoost Survival AFT

4.1 Job to be solved

Estimate when a tracked product is likely to run out for an individual or household, then send a timely reminder before depletion. The engine must distinguish a product's estimated consumption/depletion process from the customer's purchase behavior. A repeat purchase is useful evidence, but it is not proof that the previous item was empty.

4.2 Selected model: XGBoost Survival AFT

XGBoost Survival AFT is a gradient-boosted survival model for time-to-event data. Its training labels can represent an observed event time or a censored interval, rather than requiring every record to have a fully observed exact event time. This matches the customer problem better than treating every repurchase interval as an exact depletion label, provided Accesco builds labels carefully. The official XGBoost documentation describes the AFT objective, censoring bounds and supported distribution choices; see References.

What the model predicts: a conditional distribution of time until the defined event (likely depletion), given product, quantity, customer/household, purchase history, onboarding and feedback features. It should not be presented as a guaranteed run-out date.

4.3 Define the event before collecting labels

Evidence type	How it is represented	How it should be used
Confirmed depletion	Customer says “Finished” or a well-defined depletion event is otherwise directly observed; record timestamp and confidence/source.	Strongest event label, while retaining the exact meaning and potential reporting delay.
Interval-censored depletion	Last observation said “Still plenty” at time L; first observation says “Finished” at time U. True depletion lies in (L, U].	Represent an interval if supported by the label pipeline; do not arbitrarily set the event to U without documenting the approximation.
Right-censored observation	Observation ends at time C and depletion has not been observed by C.	Record that the product was not known to have run out by C; do not label C as an event.
Purchase-based proxy	A new purchase occurs after the prior purchase, but actual depletion time is unknown.	Use as weak/proxy evidence only with a clear label policy and sensitivity checks. A purchase may happen early, after running out, due to promotion, or for another household member.
Ambiguous/contaminated period	Customer may have bought elsewhere, substituted another product, shared stock, wasted stock, changed usage, or stopped using the item.	Censor, flag or exclude according to a documented rule; do not force an exact depletion date.

4.4 Why survival AFT is appropriate to the target

- The target is time until a future event, not a conventional class label such as “will purchase in the next 7 days.”
- Not every customer's depletion event will be observed; censoring is a first-class part of the training design.
- The AFT formulation models event time through a feature-dependent time scale and a selected distribution. The distributional assumption must be checked against Accesco's observed data.
- Gradient-boosted trees can use nonlinear interactions between product type, purchased quantity, household size, usage frequency, prior intervals and feedback signals.
- The model can provide a predicted time distribution/quantiles, which supports reminder windows and uncertainty-aware messaging rather than a single overconfident date.

4.5 AFT configuration and output interpretation

At implementation, configure XGBoost's `survival:aft` objective and provide lower and upper bounds for each label. For an exact event, lower and upper bounds are equal. For right-censored records, the lower bound is the observation time and the upper bound is positive infinity. Interval-censored labels use a finite lower and upper bound where the implementation supports that encoding. The exact conventions, data types and prediction transform must follow the selected XGBoost version's official documentation and be tested with unit tests.

Select the AFT error distribution and scale through time-based validation, not intuition alone. The distribution is a model assumption. Record it in the model registry. For customer-facing use, translate model outputs into calibrated depletion-time quantiles or probability of depletion by a horizon, and verify that the chosen prediction API produces the intended scale before deploying reminders.

4.6 Prediction unit and features

A prediction row should represent a **customer/household × product × prediction timestamp** (a landmark row). It uses only data known at that timestamp. A product may have several rows over time as new purchases or feedback arrive; rows from one customer/product must be split carefully during evaluation to avoid leakage.

Feature family	Example attributes	Notes
Product and pack	sku_id; category; subcategory; brand; product type; base unit; pack size; quantity per purchase; shelf life if relevant.	Product identity is essential; product attributes must be available at prediction time.
Onboarding	household size; user/household profile; usage frequency estimate; typical amount used; product usage purpose; start tracking date.	Self-reported estimates; store source and timestamp; allow unknown/prefer-not-to-answer.
Purchase history	purchase timestamp; quantity; pack size; time since last purchase; previous inter-purchase intervals; mean/median/variability of intervals; purchase count.	Inter-purchase intervals are proxy evidence, not observed depletion by default.
Consumption/remaining stock	reported remaining quantity; remaining-stock bucket; “Still plenty / Running low / Finished”; estimated daily/weekly usage if provided.	Feedback should update the feature snapshot and event/censoring status.
Household context	profile_type (personal/shared); household_id; household size; whether item is shared; member-specific usage if tracked.	Avoid merging personal and shared consumption when the tracked stock pools differ.
Behavioral context	days since purchase; prior early/late repurchase tendency; skips; repeated purchases; quantity changes; periods of inactivity.	Use only when observed; distinguish missing from zero.
Reminder history	prior reminder timestamp; reminder type; dismissed/opened; “not needed yet”; reminder snooze; resulting purchase/feedback.	Useful for policy and evaluation; avoid making response-to-reminder leakage look like natural consumption.
Calendar/context	weekday/month/season; known holiday; time since onboarding; local seasonality where relevant.	Only include features with a credible mechanism and enough evidence.
Label quality	event_type; event_lower_days; event_upper_days; censoring_type; evidence_source; label_confidence.	Label metadata belongs in training data and quality monitoring; do not blindly use target-derived values as features.

4.7 Cold-start and staged personalisation

Because Accesco does not yet have historical customer purchase and consumption records, the customer model should not be trained on fabricated or assumed customer labels. Start with a transparent bootstrap and collect data. The model can be developed in parallel using properly licensed external data only if its definitions match the target, but external data should not be assumed to transfer directly to Accesco users.

Stage	Evidence available	Prediction strategy	Promotion rule
Stage 0 — onboarding	Product/category, pack size, household size, stated usage frequency/amount; no observed purchases.	Use a global product/category prior or validated consumption estimate adjusted by onboarding; expose broad uncertainty and let user edit assumptions.	Do not claim personal model accuracy. Label result as an initial estimate.
Stage 1 — early observations	One or two purchases, intervals, optional feedback.	Blend global product/category evidence with observed personal signals; keep uncertainty broad. Do not interpret a single interval as consumption rate.	Capture feedback and monitor whether estimates are systematically early/late.
Stage 2 — 3–5 purchases	Several purchase observations for the tracked product, possibly with remaining-stock feedback.	Begin prioritising personal history if the observations are usable and reasonably consistent; keep global product priors as support.	3–5 purchases is a starting threshold, not a guarantee. Require minimum label quality/consistency checks.
Stage 3 — established history	Repeated purchases plus event/censoring labels and feedback.	Use the trained AFT model with personal and household features; return time quantiles/probabilities.	Personalisation is earned by evidence and validated calibration, not purchase count alone.
Stage 4 — drift or context change	Household size, usage, product quantity, sharing or purchase cadence changes.	Update features and reduce trust in stale personal patterns until the new pattern is observed.	Track drift; allow user correction/reset.

4.8 Reminder policy

The model provides an estimated depletion distribution; a separate notification policy decides when to remind. The policy should choose a window that gives the customer time to reorder without sending repeated or premature notifications.

- Use a conservative quantile or estimated probability of depletion within a chosen horizon to schedule the first reminder. Select the quantile and lead-in time through user research and measured outcomes.
- Suppress or defer reminders after “Still plenty,” a snooze, a recent purchase, a product pause or an explicit opt-out.
- After “Running low,” move the predicted depletion window closer and optionally ask for remaining quantity.

- After “Finished,” record a depletion event (with timestamp/source) and offer a reorder path; do not treat the click time as exact physical depletion time unless the UX defines it that way.
- Use push/in-app notifications by default. WhatsApp must be opt-in, with consent timestamp, scope, revocation and channel-level suppression.
- Limit frequency and provide controls per product/category. Log delivery, open, dismiss, snooze, feedback and resulting purchase separately.

4.9 Customer output contract

Field	Meaning
customer_id / household_id / profile_id / sku_id	Prediction entity; use pseudonymous internal identifiers in analytics.
prediction_timestamp / data_as_of	When prediction was produced and which events it knew about.
predicted_depletion_p50_days	Median time-to-depletion estimate if the prediction interface supports this interpretation.
predicted_depletion_p10_days / p90_days	Uncertainty interval/quantiles, after calibration.
prob_depletion_within_3d / 7d / 14d	Optional horizon probabilities derived correctly from the survival distribution.
evidence_stage / personal_purchase_count	Cold-start/personalisation stage and count of eligible observations.
label_or_feedback_status	Most recent stock-state response and source/time; not a model prediction.
recommended_reminder_at / reminder_reason	Policy decision and reason; distinguish it from predicted depletion time.
channel_eligibility / consent_status	Push/WhatsApp eligibility and consent record.
model_version / feature_version / uncertainty_flag	Reproducibility and safety signals.

5. Detailed data dictionary: required data and attributes

The following tables are implementation specifications. “Must-have” means required for the relevant capability or a reliable label; “strongly recommended” means the system can start without it but prediction quality or action safety will be weaker. Data availability should be profiled before any training claim is made.

5.1 Shared product and location master data

Entity / field	Priority	Definition / validation
product.sku_id	Must-have	Stable, unique SKU identifier; no reuse after product identity changes.
product.product_name	Must-have	Human-readable display name; maintain historical name changes if needed.
product.category_id / subcategory_id	Must-have	Controlled category taxonomy; version taxonomy changes.
product.brand_id	Recommended	Normalised brand identifier; distinguish unknown from not applicable.
product.base_uom	Must-have	Canonical quantity unit: piece, g, ml, etc.
product.pack_size_value + pack_size_uom	Must-have	Declared amount per sellable unit/pack; conversion to base unit must be explicit.
product.pack_to_base_factor	Must-have	Validated conversion factor used to normalise order quantities.
product.is_perishable	Must-have	Boolean classification with accountable owner.
product.shelf_life_days	Must-have for perishables	Typical shelf life; lot-level expiry overrides generic estimate.
product.storage_constraints	Recommended	Ambient/chilled/frozen or other storage requirement if applicable.
product.active_from / active_to / assortment_status	Must-have	Effective-dated catalogue availability and lifecycle.
location.location_id	Must-have	Stable ID for central warehouse and every dark store.
location.location_type	Must-have	Controlled values such as central_warehouse or dark_store.
location.region / timezone / operating_hours	Recommended	Needed for local calendar alignment, cut-offs and operations.
product_location.assortment_live_from/to	Must-have for warehouse	Whether SKU was intended to be sold at a location on each date.
product_location.capacity_limit / storage_limit	Recommended	Operational limit used by the planner.
supplier.supplier_id / sku_supplier_map	Must-have for supplier ordering	Approved supplier-to-SKU mapping, sourcing priority and effective dates.
supplier.pack_multiple / MOQ	Must-have for supplier recommendations	Supplier order constraints and unit of measure.
supplier.contract_status / order_cutoff	Recommended	Operational order eligibility and supplier cut-off rules.

5.2 Warehouse transaction and inventory tables

Entity / field	Priority	Definition / validation
order.order_id / order_created_at	Must-have	Stable transaction key and event timestamp.
order_line.order_line_id / order_id / sku_id / location_id	Must-have	Line-level identity and fulfilment location.
order_line.ordered_qty / fulfilled_qty	Must-have	Store both ordered and actually fulfilled quantity.
order_line.cancelled_qty / substituted_qty / returned_qty	Must-have where applicable	Separate demand from cancellation, substitution and returns.
order_line.unit_price / discount / promotion_id	Recommended	Price/promo data known at the time of sale.
order_line.status / fulfilment_status	Must-have	Distinguish placed, accepted, fulfilled, cancelled, returned and substituted.
inventory_snapshot.snapshot_at / sku_id / location_id	Must-have	Timestamped on-hand inventory snapshot; frequency sufficient for daily reconciliation.
inventory_snapshot.usable_qty / reserved_qty / damaged_qty	Must-have	Separate usable, reserved and unavailable stock.
inventory_movement.movement_id / type / qty / timestamp	Must-have	Receipts, dispatches, transfers, adjustments, write-offs and returns with signed quantity semantics.
inventory_movement.reference_id / source / destination	Must-have for transfers	Link movement to PO, transfer, order or adjustment and identify from/to locations.
stockout.start_at / end_at / reason	Must-have if available	Explicit out-of-stock intervals; include partial availability where possible.
purchase_order.po_id / supplier_id / sku_id / ordered_qty	Must-have for supplier planning	Order quantity and source supplier.
purchase_order.created_at / promised_at / received_at	Must-have for lead time	Actual timestamps; promised and actual arrival must remain distinct.
purchase_order.cancelled_qty / received_qty / status	Must-have	Partial receipts, cancellations and open balances.
transfer.transfer_id / sku_id / qty / from_location / to_location	Must-have for network planning	Transfer requested, dispatched, received, cancelled and quantities at each state.
transfer.requested_at / dispatched_at / received_at	Must-have for transfer lead time	Actual transit and handling time.
lot.lot_id / sku_id / received_at / expiry_at / qty	Must-have for lot-aware perishables	Lot-level stock and expiry tracking.
writeoff.qty / reason / timestamp	Recommended	Expiry, damage, shrinkage and other losses; important for effective demand and inventory policy.
supplier_lead_time_history	Must-have for production order timing	Actual lead-time observations, preferably by supplier × SKU or supplier × category; retain variance.
data_quality_flags	Must-have	Duplicate, late, corrected, missing, impossible-quantity and unit-conversion flags.

5.3 Customer, household and onboarding tables

Entity / field	Priority	Definition / validation
customer.customer_id	Must-have	Pseudonymous stable customer ID; keep direct identifiers out of model features.
household.household_id	Must-have for shared mode	Stable shared-profile ID with membership and permissions.
profile.profile_id / profile_type	Must-have	Individual or household/shared tracking profile.
profile.member_id / role / active_from/to	Recommended	Household membership, role and effective dates; permission to edit shared stock.
tracking_item.tracking_id / profile_id / sku_id	Must-have	Explicit link between a profile and a tracked product.
tracking_item.is_shared	Must-have	Distinguishes shared household stock from personal stock.
onboarding.household_size	Recommended	Number of people whose use is covered by the tracked stock; allow unknown.
onboarding.usage_frequency	Must-have for bootstrap if provided	Self-reported usage cadence, with controlled units/options and source timestamp.
onboarding.typical_quantity_used	Recommended	Self-reported amount per day/week/use; record unit and confidence.
onboarding.product_use_context	Optional	Context that changes consumption, only if justified and consented.
onboarding.started_tracking_at	Must-have	Start of observation; necessary to calculate elapsed/censored time.
onboarding.answer_source / updated_at	Must-have	Self-reported values can change; retain history rather than overwrite silently.
notification_consent.channel / scope / granted_at / revoked_at	Must-have for WhatsApp	Explicit channel consent, purpose, evidence and revocation.
customer_timezone / preferred_reminder_window	Recommended	Respect local time and user preferences.

5.4 Customer purchase, feedback and reminder data

Entity / field	Priority	Definition / validation
customer_order.order_id / customer_id / household_id	Must-have	Purchaser/profile identity; support purchases for personal and shared profile.
customer_order_line.sku_id / qty / unit / pack_size	Must-have	Normalised purchased quantity, not just line count.
customer_order.ordered_at / paid_at / fulfilled_at / cancelled_at	Must-have	Use a consistent event definition for “purchase”; retain all timestamps.
customer_order.source	Must-have	Accesco app vs imported/manual entry vs known external purchase, if captured.
feedback.feedback_id / profile_id / sku_id / timestamp	Must-have when feedback used	Timestamped stock-state response tied to the tracked product.
feedback.stock_state	Must-have	Controlled values: still_plenty, running_low, finished, unknown.
feedback.remaining_qty / remaining_qty_unit	Strongly recommended	Optional exact remaining quantity; validate units and non-negative values.
feedback.last_known_plenty_at / first_finished_at	Must-have for interval labels where collected	Provides interval bounds for when depletion likely occurred.
feedback.source / confidence / correction	Must-have	Distinguish user report from inference and allow correction.
reminder.reminder_id / scheduled_at / sent_at / channel	Must-have	Record planned and actual send time.
reminder.delivery_status / opened_at / dismissed_at / snoozed_until	Recommended	Notification delivery and response outcomes.
reminder.reason / model_version / prediction_id	Must-have	Connect action to the prediction and policy decision.
reminder.resulting_purchase_id / feedback_id	Recommended	Outcome link for evaluation; do not assume causation.
product_pause / tracking_stop / opt_out timestamps	Must-have	Define observation end and avoid reminders after stop/opt-out.

5.5 Training-table schemas

Warehouse daily panel — one row per date × SKU × location:

```
business_date, sku_id, location_id, fulfilled_qty_base_uom, ordered_qty_base_uom, cancelled_qty, returned_qty,
in_stock_duration, stockout_flag, opening_usable_qty, closing_usable_qty, reserved_qty, receipts_qty,
transfer_in_qty, transfer_out_qty, writeoff_qty, price, discount, promotion_flag, holiday/event flags,
assortment_live, data_quality_flags
```

Customer survival landmark table — one row per profile × tracked SKU × prediction/landmark timestamp:

```
profile_id, profile_type, household_id, sku_id, prediction_at, days_since_tracking_start, purchase_count,
last_purchase_qty_base_uom, days_since_last_purchase, interpurchase_interval_summary, onboarding_usage_features,
household_size, is_shared, last_stock_state, remaining_qty, prior_reminder_features, event_lower_days,
event_upper_days, censoring_type, evidence_source, label_confidence
```

These are logical schemas, not mandatory physical table names. Build stable primary keys, foreign keys, effective dates, units and data lineage around them.

6. Event definitions, labels and feature engineering

6.1 Warehouse target construction

- Define fulfilled demand at the SKU × location × business-day level. Keep cancellations, substitutions, returns and stockout constraints separately visible.
- If demand is censored by stock availability, use availability indicators and a documented correction/estimation policy; never assume observed sales always equal unconstrained demand.
- Treat negative inventory adjustments, late events and corrections as ledger facts with explicit reasons. Rebuild the daily panel from the canonical ledger where possible.
- Generate lag/rolling features only from data available before the forecast timestamp. With TimesFM, the main input is the historical series; covariates must be aligned and known at forecast time.
- Include assortment-live flags and store opening dates so that a newly opened store or newly listed SKU is not treated as having a long history of zero demand.

6.2 Customer survival target construction

For each landmark timestamp, the target is elapsed time from that landmark to the next observed/estimated depletion event or the end of reliable observation. If the depletion event occurs after the landmark, encode the elapsed time and censoring bounds in the format expected by XGBoost AFT. If an observation period is cut off while the item is still not known to be depleted, the record is right-censored, not a completed event.

Situation	Label rule	Do not do this
Customer explicitly confirms finished	Use the defined event timestamp or a bounded interval if reporting delay is material; preserve source/confidence.	Assume the tap time is exact physical depletion without defining it in the UX.
Still plenty at day 8, finished reported at day 11	Represent depletion as interval-censored between the observations when supported by the pipeline.	Pretend depletion occurred exactly on day 11 without recording the approximation.
No finished response by day 20; tracking ends	Right-censor at day 20, assuming the last known state supports no observed event by then.	Label day 20 as a depletion event.

Situation	Label rule	Do not do this
New purchase on day 18, no stock feedback	Mark as purchase-based proxy/ambiguous; apply a documented weak-label policy or leave depletion censored/unknown.	Automatically label depletion as day 18.
Customer buys a second pack while first pack may remain	Record purchase and quantity; use remaining-stock feedback if possible; treat as potential overlap.	Assume old pack was empty because a new pack was purchased.
Customer switches brand/product or buys elsewhere	Capture substitution/external purchase if known; otherwise flag uncertainty or censor.	Treat the gap as normal use of the tracked SKU.

6.3 Avoiding leakage and repeated-row bias

- At each prediction timestamp, compute features using only prior events; a feedback response that later confirms “Finished” cannot be used as a feature in an earlier prediction.
- Use time-based validation. Keep later observations out of training and avoid random row splits that allow the same customer-product trajectory to leak across train and test.
- Group by customer/household and tracked product when checking generalisation. Evaluate both known users and new-user cold-start scenarios.
- Avoid letting a large number of rows from one highly active customer dominate the objective without inspection. Consider weighting or sampling that reflects the intended deployment population.
- Keep weak purchase-proxy labels distinguishable from direct depletion labels and report metrics separately by evidence quality.

7. Cold-start, feedback and household handling

7.1 Global product evidence

The global prior should be product-specific where data supports it, then category-level where product-specific evidence is sparse. External data may seed an initial range, but its product definitions, units, population, geography and measurement method must be documented. Do not hard-code a simple linear rule such as “double the quantity means double the duration.” Learn quantity-duration relationships from observed data and feedback, and preserve uncertainty.

7.2 Household versus personal stock

A shared household profile represents one shared stock pool. Household size and member usage can affect depletion, but the system should not automatically multiply consumption by household size using a fixed linear formula. A personal profile represents stock allocated to one individual. If a household tracks both shared and personal items, each tracking record must specify which pool it represents. Shared profiles need permissions, clear ownership and controls to prevent conflicting feedback from multiple members.

7.3 Feedback UX and semantics

Feedback choice	Interpretation	Suggested model/data effect
Still plenty	The item is not currently near depletion, according to the reporter.	May establish a lower bound on time-to-depletion; suppress/shift reminders and record observation time.
Running low	Depletion is approaching but not necessarily immediate.	Update remaining-stock state and reminder timing; if no exact amount is supplied, treat as an ordinal state rather than an exact quantity.
Finished	Reporter says the item is depleted now or was depleted recently.	Record event report time, possible reporting delay and source; use a defined event/interval label.
Remaining quantity	Optional quantitative stock estimate in a known unit.	Convert to base UOM; validate; use as a feature and potentially a stronger stock-state observation.
Not using / paused	Consumption has stopped or tracking is inactive.	Pause predictions/reminders or mark a distinct competing state; do not continue normal depletion assumptions.

7.4 Notification safeguards

- Store consent separately from delivery preferences; revocation must immediately suppress the opted-out channel.
- Provide per-product pause, snooze, dismiss and stop-tracking controls.
- Do not infer that a reminder caused a purchase simply because a purchase followed it. Treat reminder exposure as context and evaluate policy effects separately.
- Avoid sending a reminder if a recent purchase, reported remaining stock, product pause or known external purchase makes it stale.
- Use uncertainty-aware language such as “You may be running low” instead of “You will run out tomorrow.”

8. Outputs, APIs, workflow and human controls

8.1 Daily warehouse workflow

1. Ingest and reconcile prior-day order lines, inventory movements, receipts, transfers, stock snapshots and supplier updates.

2. Run data-quality checks: missing SKU/location keys, duplicate events, unit mismatches, impossible stock balances, late events and unexplained demand zeros.

3. Build/update the SKU × location daily demand series and availability/stockout flags.

4. Generate forecasts and uncertainty estimates for the required horizon.

5. Calculate inventory position, lead-time demand, safety stock and replenishment need using approved policies.

6. Apply expiry, MOQ/pack, capacity, supplier eligibility and transfer constraints.

7. Rank exceptions and recommendations; show rationale, uncertainty, stock coverage and risk.

8. Require authorized staff approval; record edits/rejections and reason codes.

9. Track recommendation outcome, stockout/overstock/spoilage and forecast error for continuous evaluation.

8.2 Customer prediction workflow

1. When a product is tracked, capture onboarding values, profile type, quantity/unit and consent/preferences.

2. Create an initial estimate from product/category evidence adjusted by the onboarding inputs; label it as an estimate.

3. When purchases, feedback or tracking changes arrive, update the customer-product observation state and event log.

4. Build a prediction-time feature snapshot without future information; calculate the correct survival/censoring target later for training.

5. Use XGBoost Survival AFT once training data is adequate and the model has passed temporal, calibration and subgroup validation; until then, keep the bootstrap policy.

6. Convert the predicted time distribution into a reminder decision using approved lead-in, confidence and frequency rules.

7. Check recent purchase, feedback, pause/opt-out and channel consent before sending.

8. Log prediction, scheduled reminder, actual delivery, user response and subsequent outcome for evaluation.

8.3 Human control and audit requirements

Control	Warehouse	Customer-facing
Explainability	Show demand forecast, inventory position, lead time, safety stock, constraints and reason codes.	Show estimated depletion window, recent evidence, uncertainty and why a reminder is proposed.
Override	Approve, edit quantity/date/source, reject; reason required for material overrides.	User can dismiss, snooze, correct stock state, pause or stop tracking.
Audit	Approver, timestamps, model/planner version, input snapshot, action and outcome.	Prediction/model version, consent state, reminder channel, delivery and response events.
Fallback	If forecast/data is invalid, create an exception for staff rather than a precise order recommendation.	If prediction is invalid/low confidence, use a broad editable estimate or suppress reminder.
Safety boundary	No auto-placing supplier orders under this design; staff approval is mandatory.	No WhatsApp without explicit opt-in; do not represent predictions as certainty.

9. Data quality, validation, monitoring and retraining

9.1 Warehouse validation

Validation area	Metrics/checks	Decision use
Forecast accuracy	WAPE, MAE, RMSE and bias by SKU, category, location and horizon; use metrics suitable for zero/intermittent demand.	Determine whether forecasts are usable across the actual product mix; do not rely on one aggregate score.
Probabilistic quality	Quantile/interval coverage and width; calibration by horizon and demand segment.	Check that uncertainty estimates can support service-level decisions.
Operational outcomes	Stockout rate/duration, fill rate, on-time availability, average inventory, turns, spoilage/write-off, emergency transfers.	Measure whether forecast + planner improves outcomes, not only mathematical accuracy.
Decision cost	Weighted stockout cost, holding cost, expiry/waste and handling/transfer costs where credible values exist.	Set policies with Operations/Finance and monitor trade-offs.
Data health	Missing demand days, stockout contamination, late events, reconciliation differences, unit mismatch, feature staleness.	Block or downgrade recommendations when source data is unreliable.
Segment performance	Fast/slow movers, new SKUs, perishable/non-perishable, new stores, promotion periods.	Identify where a forecast should be flagged or policy adjusted.

9.2 Customer survival validation

Validation area	Metrics/checks	Decision use
Survival discrimination	Concordance index or time-dependent discrimination metrics where appropriate.	Check ranking of earlier versus later depletion risk.
Calibration	Predicted versus observed event rates by horizon; calibration curves for depletion by 3/7/14 days; survival probability calibration.	Most important for choosing reminder timing.
Time error	Error on high-quality observed depletion labels; interval-aware metrics where available.	Quantify timing accuracy without treating weak purchase proxies as truth.
Censoring/label audit	Share of direct events, interval-censored, right-censored, proxy and ambiguous records; audit label sources.	Avoid training on untrustworthy targets.
Reminder outcomes	Premature-reminder rate, missed-depletion rate, dismissal/snooze, opt-out, user-reported usefulness, purchase after reminder (descriptive only).	Tune reminder policy while accounting for notification effects and user preference.
Cold-start quality	Metrics by purchase-count stage, onboarding-only users, product/category and household/personal profile.	Confirm that the global prior and personalisation transition work for new and returning users.
Subgroup and drift	Error/calibration by household size, category, quantity bands, product lifecycle and time since last update.	Find systematic bias and detect changing consumption patterns.

9.3 Splitting and backtesting

- Warehouse: use rolling-origin backtests. Train on past dates and forecast later dates, preserving the operational information set. Evaluate multiple horizons and seasonal periods where enough data exists.
- Customer: use temporal splits and ensure event/censoring construction is identical in training and evaluation. Keep later observation windows out of training. Report results separately for direct labels versus weak proxies.
- For both engines, compare against a transparent operational baseline during evaluation even though this document does not recommend an alternative production model. The purpose is to prove that the selected model adds value rather than to publish a model comparison exercise.
- Do not declare production readiness from a single random train/test split or from metrics calculated on synthetic data alone.

9.4 Monitoring and retraining triggers

Trigger	Warehouse response	Customer response
Data pipeline failure / stale feed	Block/downgrade affected recommendations; alert data owner.	Pause affected predictions/reminders or use clearly marked bootstrap.
Forecast/calibration drift	Re-run backtest, inspect assortment/stockout changes, update model/policy if validated.	Re-check calibration by product/profile stage; retrain only after label quality and volume are adequate.
Product or pack-size change	Version catalogue attributes and avoid mixing incompatible history without conversion.	Update quantity units; reset/discount stale personal patterns if consumption semantics changed.
Supplier or transfer lead-time change	Update lead-time estimates and planner policy; forecast model may not need retraining solely for lead-time changes.	Not directly relevant unless delivery/availability affects customer experience.
Household/usage change	Not usually a warehouse model event except aggregate demand shifts.	Update profile and reweight recent evidence; allow pause/reset.
New SKU/location/user	Use cold-start policy and flag low confidence.	Use product/category prior + onboarding; do not claim personal accuracy.

10. Implementation plan, risks and acceptance criteria

10.1 Phased implementation

Phase	Build work	Exit criteria
Phase 0 — Data contracts	Confirm canonical IDs, units, event names, timestamp semantics, source owners, warehouse locations, household/profile semantics, consent handling.	Reviewed schemas; data owners assigned; source-to-canonical mapping documented.
Phase 1 — Instrumentation	Capture order lines, stock ledger, POs, receipts, transfers, stockouts, customer purchases, onboarding, feedback and reminders.	Events are idempotent, timestamped, unit-consistent and auditable; missing fields measured.
Phase 2 — Baseline operations	Build daily warehouse panel, inventory position, rule-based planner and staff approval UI. Build customer onboarding/global-prior estimates and feedback/reminder flow.	Staff can inspect/approve recommendations; customer can edit/pause/feedback; no model training dependency.
Phase 3 — Shadow evaluation	Run TimesFM forecasts without controlling orders; build customer survival labels and run AFT in offline/shadow mode when adequate labels exist.	Rolling backtests and temporal validation; data/label audit; model versions logged.
Phase 4 — Controlled rollout	Expose warehouse suggestions to staff with approval; gradually enable customer reminders for eligible users and measure usefulness/opt-outs.	Operational sign-off; calibration thresholds met; rollback and fallback tested.
Phase 5 — Continuous improvement	Monitor forecast and survival calibration, inventory outcomes, notification quality and data drift; retrain under governed release process.	Scheduled model review; retraining criteria; audit and change log in place.

10.2 Key risks and mitigations

Risk	Impact	Mitigation
No historical operational data	No credible Accesco-specific model training at launch.	Build collection first; use transparent rules/bootstrap; label estimates provisional.
Sales confused with demand	Stockouts or assortment gaps can bias forecasts downward.	Record in-stock duration and stockout intervals; audit censored demand periods.
Supplier lead-time history missing	Order dates/quantities may be poorly timed.	Collect promised and actual timestamps; begin with explicitly provisional lead-time assumptions approved by Operations.
Repurchase mistaken for depletion	Customer reminders become premature or late; labels become noisy.	Use feedback and interval/right censoring; keep purchase proxies distinct and measure label quality.
Weak/biased external priors	Cold-start estimates may not transfer to Accesco users.	Record source and units; expose uncertainty; update from local data and user feedback.
Household stock ambiguity	Shared and personal consumption may be mixed.	Explicit profile_type/is_shared; clear ownership and member permissions.
TimesFM-3 deployment license	Production deployment may breach checkpoint terms.	Verify exact checkpoint, license and serving route before production; prefer a commercially permitted deployment.
Overconfident outputs	Bad stock decisions or customer trust loss.	Calibrate uncertainty, show caveats, define fallback thresholds and maintain human controls.
Notification fatigue/privacy	Opt-outs, complaints and loss of trust.	Explicit channel consent, frequency limits, snooze/pause controls and data minimisation.

10.3 Minimum acceptance criteria before production

- All key entities have stable IDs, normalized units and reliable event timestamps; duplicate and late-event handling is tested.
- Warehouse order/stock/transfer/receipt reconciliation is explainable, and forecast evaluation includes stockout periods and intermittent-demand segments.
- The warehouse planner's service-level, safety-stock, MOQ, shelf-life, capacity and approval policies are documented and signed off by Operations.
- The selected TimesFM checkpoint/deployment route is approved for intended commercial use and meets latency/cost/operational requirements.
- Customer AFT labels distinguish direct, interval-censored, right-censored, proxy and ambiguous evidence; training data leakage tests pass.
- Customer predictions are calibrated enough for the chosen reminder policy; high-uncertainty outputs trigger broader estimates or suppression.
- The 3–5 purchase threshold is implemented as an initial policy with quality/consistency checks, not a hard guarantee of personalization accuracy.
- Household/shared versus personal tracking works; consent and revocation tests pass; WhatsApp remains disabled until explicit opt-in is recorded.
- Prediction logs, model versions, fallback paths, user/staff overrides and outcome measurement are available.

11. References and implementation notes

The technical descriptions below are grounded in official model documentation. The architecture, data dictionary and operational policies are design recommendations tailored to the stated Accesco constraints and must be confirmed with system owners before implementation.

1. XGBoost — Survival Analysis with Accelerated Failure Time
https://xgboost.readthedocs.io/en/stable/tutorials/aft_survival_analysis.html
Official explanation of time-to-event regression, censoring, label bounds and the survival:aft objective.

2. Google Research — TimesFM repository
https://github.com/google-research/timesfm
Official repository and release notes for TimesFM; includes version and licensing notes for checkpoints.

3. Google Cloud — The TimesFM model in BigQuery ML
https://docs.cloud.google.com/bigquery/docs/timesfm-model
Official documentation for the managed TimesFM implementation and commercial-use context.

4. XGBoost — Parameter reference
https://xgboost.readthedocs.io/en/stable/parameter.html
Official objective configuration reference; confirm the installed XGBoost version and AFT parameters at implementation.

Appendix A. Practical implementation checklist

■ Confirm business definitions: fulfilled demand, purchase event, depletion event, observation end, usable stock, reserved stock, transfer lead time.

■ Confirm IDs and unit conversion rules for products, packs, locations, suppliers, customers, households and tracking items.

■ Instrument warehouse events: order lines, stock snapshots/movements, stockout intervals, purchase orders, receipts, transfers, lots and expiry.

■ Instrument customer events: onboarding, purchase quantity, feedback state/remaining quantity, reminders, pause/stop, channel consent and opt-out.

- Create data-quality dashboards before model training.
- Build warehouse planner with staff approval and customer cold-start estimate flow.
- Implement training-row generation with event-time features and survival censoring bounds.
- Run time-based backtests, label audits and calibration checks.
- Verify TimesFM licensing for the actual deployment path.
- Define model release, rollback, retraining and human-override procedures.